SLIIT | FACULTY OF COMPUTING | DEPARTMENT OF SOFTWARE ENGINEERING

SE3090 – Software Engineering Frameworks

Lab Practical 02

Advanced React Frontend Development

Lab Practical Number	Lab 02
Lab Title	Advanced React Frontend Development
Related Lecture	Lecture 02 – Advanced React Frontend Development
Duration	2 Hours (in-lab)
Learning Outcomes	LO2 – Apply frameworks & tools • LO3 – Best practices for quality • LO4 – Select & justify approaches
Mode	Individual coding • Word document + project ZIP submitted to CourseWeb within the session

Introduction

This lab turns Lecture 02 into working code. In the lecture you studied production-grade React: component-based architecture, a feature-based folder structure, reusable components and custom hooks, controlled forms with validation, client state versus server state, data fetching with loading and error handling, and React Router navigation. In this 2-hour session you will build a small but realistic single-page application that exercises each of these ideas.

Everything you need comes from Lecture 02. Base your implementation and explanations on the lecture content, the worked LankaMart case study, and the class examples (useDebounce, useFetch, controlled inputs, the three request states, nested/lazy routes). You may add small examples of your own, but stay within the scope of Lecture 02.

Lab Objectives

By the end of this lab, you should be able to:

- **Scaffold a React app and organise it** with a feature-based folder structure and clear separation of UI, hooks, and services (LO2, LO3).
- **Build reusable, composable components** that receive data through props and render dynamic lists with keys and conditional UI (LO2).
- **Manage state with hooks** using useState and useEffect, and apply a custom useDebounce hook for an efficient search (LO2, LO3).
- **Fetch data and handle all three request states** — loading, success, and error — through a reusable useFetch hook (LO2, LO3).
- **Add routing, navigation, and a validated form** with React Router and controlled inputs that show per-field errors (LO2, LO3).
- **Select and justify state-management tools** for each kind of state using the lecture's decision table (LO4).

Required Tools / Software

- **Node.js (LTS) and npm —** verify with node -v and npm -v in a terminal.
- **Visual Studio Code —** (or any editor) for writing React components.
- **A modern web browser —** Chrome or Firefox; the React DevTools / Profiler extension is recommended (Lecture 02, Part 5).
- **Vite —** the build tool used to scaffold the React app (per the lecture's preparation slide).
- **react-router-dom —** for Task 05 routing; install inside the project with npm.
- **Optional: axios, react-hook-form, zod —** for the lecture-aligned enhancements noted in Tasks 04 and 05.

> **TECHNICAL NOTE**
>
> This lab uses JavaScript (.jsx) so you can finish in two hours. The lecture recommends **Vite + React + TypeScript** for production — using TypeScript is welcome if you are comfortable with it, but it is not required for marks.

Lab Time Plan (120 minutes)

Time	Activity	Output
0:00 – 0:10	Briefing & recap of Lecture 02 (architecture, hooks, state, routing)	Ready to code
0:10 – 0:25	Task 01 — Scaffold the app + feature-based folders	Running app + structure
0:25 – 0:45	Task 02 — Reusable components, props, list & conditional rendering	Menu grid renders
0:45 – 1:05	Task 03 — State, useDebounce hook, events (live search)	Working search filter
1:05 – 1:30	Task 04 — Data fetching with loading / success / error states	Data loads from JSON
1:30 – 1:50	Task 05 — Routing, navigation, validated form + LO4 justification	Multi-page app + form
1:50 – 2:00	Quick test + finalise, screenshot, zip and submit	Submitted .docx + .zip

> **HOW TO USE THIS PLAN**
>
> These times are a guide. Keep Tasks 01–02 brisk; protect time for Tasks 04–05, which carry the most marks. If you fall behind, get the app running and the menu listing working first, then add search, fetching, and routing in order.

Mini Project Scenario

"CAMPUSEATS" — MENU & ORDERS DASHBOARD

You are building the **frontend** for CampusEats, a campus food-ordering service. Students browse a menu of dishes, search for a dish, open a dish to see its details, and place a simple order through a validated form.

In production this React app would call the ASP.NET Core REST API you build in Lectures 03–04. For this lab the data comes from a local JSON file so you can focus on the frontend.

What you will build

- A `MenuPage` that lists dishes as reusable `DishCard` components.
- A debounced search box that filters the menu as the user types.
- Data loaded from `/menu.json` with loading, error, and success states.
- Routes for the menu, a dish detail page (`/dish/:id`), and an order form (`/order`).
- A validated order form that shows per-field errors and a success message.

Target feature-based structure (Lecture 02, Part 1)

```
src/
  components/        // shared, reusable UI (Button, Spinner)
  features/
    menu/
      components/    // DishCard, MenuList, SearchBar
      hooks/         // useDebounce, useFetch
      services/      // menuApi (and menu.json in /public)
      pages/         // MenuPage, DishDetailPage, OrderPage
  routes/            // AppRoutes
  App.jsx   main.jsx
```

TASK 01 • 15 minutes • LO2, LO3

Scaffold the React App & Feature-Based Structure

Create a fresh React app with Vite and set up the feature-based folders from the lecture. Grouping by domain (not by file type) is the architecture decision that keeps a growing codebase maintainable.

Step-by-step instructions

1. Open a terminal and create the app (choose the `React` + `JavaScript` template):

```
Terminal
$ npm create vite@latest campuseats-dashboard -- --template react
$ cd campuseats-dashboard
$ npm install
$ npm run dev
```

2. Open the printed `http://localhost:5173` URL to confirm the app runs.
3. Inside `src/`, create the feature-based folders shown in the Mini Project Scenario above.
4. In `/public`, create a `menu.json` file (you will use it in Task 04).
5. Clean up the default `App.jsx` so it renders a simple heading such as “CampusEats Dashboard”.

EXPECTED OUTPUT

The Vite dev server runs and the browser shows your CampusEats heading. A screenshot of the running app and a screenshot (or tree) of the feature-based folder structure.

TASK 02 • 20 minutes • LO2, LO3

Reusable Components, Props & List Rendering

Build a presentational `DishCard` component that receives a dish through props, and a `MenuList` that maps an array of dishes to cards. This is component-based architecture and unidirectional data flow in practice.

Step-by-step instructions

1. Add this sample data near the top of your `MenuPage` (you will replace it with a fetch in Task 04):

src/features/menu/pages/MenuPage.jsx (data)

```
const SAMPLE = [
  { id: 1, name: "Kottu Roti",   price: 750, category: "Mains",  available: true  },
  { id: 2, name: "Fried Rice",   price: 850, category: "Mains",  available: true  },
  { id: 3, name: "Veg Sub",      price: 600, category: "Snacks", available: false },
  { id: 4, name: "Watalappan",   price: 350, category: "Dessert",available: true  },
  { id: 5, name: "Iced Milo",    price: 300, category: "Drinks", available: true  },
];
```

2. Create `DishCard.jsx` — pure presentation, receives one `dish` prop, uses conditional rendering for a “Sold out” badge:

src/features/menu/components/DishCard.jsx

```
function DishCard({ dish }) {
  return (
    <div className="dish-card">
      <h3>{dish.name}</h3>
      <p>Rs. {dish.price.toFixed(2)}</p>
      <small>{dish.category}</small>
      {!dish.available && <span> — Sold out</span>}
    </div>
  );
}
export default DishCard;
```

3. Create `MenuList.jsx` — maps the array to cards with a unique `key`, and handles the empty case:

src/features/menu/components/MenuList.jsx

```
import DishCard from "./DishCard";

function MenuList({ dishes }) {
  if (!dishes.length) return <p>No dishes match your search.</p>;
  return (
    <div className="menu-grid">
      {dishes.map((dish) => (
        <DishCard key={dish.id} dish={dish} />
      ))}
    </div>
  );
}
export default MenuList;
```

4. Render `<MenuList dishes={SAMPLE} />` from `MenuPage` and confirm all five dishes appear.

EXPECTED OUTPUT

A grid (or list) of five dish cards rendered from the array via `props`. The “Veg Sub” card shows the “Sold out” label (conditional rendering), and there are no missing-key warnings in the console.

TASK 03 • 20 minutes • LO2, LO3

State, the useDebounce Hook & Event Handling

Add a controlled search box (React state is the single source of truth) and a custom `useDebounce` hook so you do not filter on every keystroke. This is the exact hook from Lecture 02, Part 2.

Step-by-step instructions

1. Create the `useDebounce` hook:

src/features/menu/hooks/useDebounce.js

```
import { useState, useEffect } from "react";

export function useDebounce(value, delay = 400) {
  const [debounced, setDebounced] = useState(value);
  useEffect(() => {
    const id = setTimeout(() => setDebounced(value), delay);
    return () => clearTimeout(id);   // cleanup
  }, [value, delay]);
  return debounced;
}
```

2. In `MenuPage`, add controlled search state and debounce it, then filter the list:

src/features/menu/pages/MenuPage.jsx

```
import { useState } from "react";
import { useDebounce } from "../hooks/useDebounce";
import MenuList from "../components/MenuList";

function MenuPage() {
  const [query, setQuery] = useState("");
  const debounced = useDebounce(query, 400);

  const filtered = SAMPLE.filter((d) =>
    d.name.toLowerCase().includes(debounced.toLowerCase())
  );

  return (
    <div>
      <input
        value={query}
        onChange={(e) => setQuery(e.target.value)}
        placeholder="Search dishes..."
      />
      <MenuList dishes={filtered} />
    </div>
  );
}
export default MenuPage;
```

3. Type in the box and confirm the list narrows. Notice the filter updates only after you stop typing (the debounce delay).

EXPECTED OUTPUT

A working search box that filters the dish list. The input is controlled (its value comes from state), and filtering is debounced so it does not run on every keystroke.

TASK 04 • 25 minutes • LO2, LO3

Data Fetching with Loading, Success & Error States

Replace the hard-coded array with data fetched from `/menu.json`. Wrap the fetch in a reusable `useFetch` hook that returns `{ data, isLoading, error }` — and render all three states, as the lecture insists.

Step-by-step instructions

1. Put the Task 02 array (as JSON) into `/public/menu.json` so it is served at `/menu.json`.
2. Create the `useFetch` hook:

src/features/menu/hooks/useFetch.js

```
import { useState, useEffect } from "react";

export function useFetch(url) {
  const [data, setData] = useState(null);
  const [isLoading, setIsLoading] = useState(true);
  const [error, setError] = useState(null);

  useEffect(() => {
    let active = true;
    setIsLoading(true);
    fetch(url)
      .then((res) => {
        if (!res.ok) throw new Error("HTTP " + res.status);
        return res.json();
      })
      .then((json) => { if (active) setData(json); })
      .catch((err) => { if (active) setError(err.message); })
      .finally(() => { if (active) setIsLoading(false); });
    return () => { active = false; };   // cleanup
  }, [url]);

  return { data, isLoading, error };
}
```

3. Use the hook in `MenuPage` and render the three states explicitly:

src/features/menu/pages/MenuPage.jsx (states)

```
const { data: dishes, isLoading, error } = useFetch("/menu.json");

if (isLoading) return <p>Loading menu…</p>;        // 1. loading
if (error)     return <p>Could not load menu: {error}</p>; // 2. error
// 3. success → render search + <MenuList dishes={...} />
```

4. Test the error path: temporarily fetch `"/wrong.json"` and confirm the error message shows, then change it back.

EVERY FETCH HAS THREE STATES

The lecture's rule: design the UI for loading, success, and error every time. A blank screen while data loads, or a silent failure, is a bug — not a missing feature.

EXPECTED OUTPUT

The menu now loads from `/menu.json`. You can show a brief loading message, the rendered list on success, and a friendly error message when the URL is wrong. Search from Task 03 still works on the fetched data.

LECTURE-ALIGNED ENHANCEMENT (OPTIONAL)

In production the lecture recommends **TanStack Query** (`useQuery`) for server state — it adds caching, refetching, and dedupe for free. If you finish early, read its docs and note in your write-up why a query library beats a hand-written `useFetch` at scale.

TASK 05 • 20 minutes • LO2, LO3, LO4

Routing, Navigation & a Validated Form

Add React Router so the app has real pages, then build a validated order form. Finally, justify your state choices using the lecture's decision table (LO4).

Part A — Routing & navigation

1. Install the router: `npm install react-router-dom`.
2. Wrap the app in `<BrowserRouter>` in `main.jsx`, then define routes and a nav bar in `App.jsx`:

src/App.jsx

```
import { Routes, Route, NavLink } from "react-router-dom";
import MenuPage from "./features/menu/pages/MenuPage";
import DishDetailPage from "./features/menu/pages/DishDetailPage";
import OrderPage from "./features/menu/pages/OrderPage";

export default function App() {
  return (
    <div>
      <nav>
        <NavLink to="/">Menu</NavLink> |
        <NavLink to="/order">Place Order</NavLink>
      </nav>
      <Routes>
        <Route path="/" element={<MenuPage />} />
        <Route path="/dish/:id" element={<DishDetailPage />} />
        <Route path="/order" element={<OrderPage />} />
        <Route path="*" element={<p>404 — Page not found</p>} />
      </Routes>
    </div>
  );
}
```

3. In `DishDetailPage`, read the URL parameter with `useParams()` and show that dish's details (plus a link back to the menu).

src/features/menu/pages/DishDetailPage.jsx

```
import { useParams, Link } from "react-router-dom";

function DishDetailPage() {
  const { id } = useParams();   // from /dish/:id
  return (
    <div>
      <p>Showing details for dish #{id}</p>
      <Link to="/">← Back to menu</Link>
    </div>
  );
}
export default DishDetailPage;
```

Part B — A validated order form

4. Build `OrderPage` with controlled inputs and a `validate()` function that returns per-field errors:

src/features/menu/pages/OrderPage.jsx

```
import { useState } from "react";

function OrderPage() {
  const [form, setForm] = useState({ name: "", email: "", qty: 1 });
```

```
  const [errors, setErrors] = useState({});
  const [done, setDone] = useState(false);

  function validate(v) {
    const e = {};
    if (v.name.trim().length < 2) e.name = "Name too short";
    if (!/^[^@\s]+@[^@\s]+\.[^@\s]+$/.test(v.email))
      e.email = "Enter a valid email";
    if (Number(v.qty) < 1) e.qty = "Qty must be ≥ 1";
    return e;
  }

  function handleChange(e) {
    setForm({ ...form, [e.target.name]: e.target.value });
  }
  function handleSubmit(e) {
    e.preventDefault();
    const found = validate(form);
    setErrors(found);
    if (Object.keys(found).length === 0) setDone(true);
  }

  if (done) return <p>Thanks, {form.name}! Order received.</p>;
  return (
    <form onSubmit={handleSubmit}>
      <input name="name" value={form.name} onChange={handleChange} />
      {errors.name && <span> {errors.name}</span>}
      <input name="email" value={form.email} onChange={handleChange} />
      {errors.email && <span> {errors.email}</span>}
      <input name="qty" type="number" value={form.qty}
             onChange={handleChange} />
      {errors.qty && <span> {errors.qty}</span>}
      <button type="submit">Place order</button>
    </form>
  );
}
export default OrderPage;
```

> **LECTURE-ALIGNED ENHANCEMENT (OPTIONAL)**
>
> The lecture's production approach is **React Hook Form + Zod**. If you finish early, install `react-hook-form zod @hookform/resolvers` and rebuild the form with a Zod schema and `zodResolver` — fewer re-renders and declarative, cross-field validation.

Part C — Justify your state choices (LO4)

Using the lecture's state-management decision table, complete the table below for the CampusEats app. Name the tool you would use for each piece of state and give a one-line reason.

Piece of state	Tool you would use	Why (one line)
Search box text		
Menu data from the API		
Logged-in user (shared app-wide)		
A cart of selected dishes		

EXPECTED OUTPUT

Working navigation between the menu, a dish detail page (reading the :id parameter), and the order form; a 404 for unknown URLs. The form blocks submission while showing per-field errors and confirms success on valid input. A completed state-justification table.

QUICK TEST • 10 minutes

Knowledge Check

Answer all questions in your Word document. Write the question number and your answer. Q1–Q5 are multiple choice; Q6 is short answer; Q7 is code-based; Q8 is scenario-based.

Multiple choice (choose one)

Q1. Data fetched from your REST API is which kind of state?

(a) local UI state
(b) server state
(c) form state
(d) route state

Q2. Best default tool for caching API data with loading and error states built in:

(a) Redux
(b) Context API
(c) TanStack Query
(d) useState

Q3. A child wrapped in React.memo still re-renders. The most likely cause is:

(a) a React bug
(b) an unstable (newly created) prop
(c) too much CSS
(d) a missing key

Q4. Which hook gives a STABLE function reference between renders?

(a) useMemo
(b) useEffect
(c) useCallback
(d) useRef

Q5. A frontend route guard hides /admin from the menu. Is the data secure?

(a) yes, always
(b) only if the API enforces authorization
(c) yes, if the route is lazy-loaded
(d) yes, if you use Context

Short answer

Q6. In a feature-based folder structure, what is the rule for grouping files, and which anti-pattern does it avoid?

Code-based

Q7. Complete this controlled input so React state is the single source of truth:

```
const [query, setQuery] = useState("");
<input  ______  onChange={(e) => setQuery(______)} />
```

Scenario-based

Q8. For each piece of state, name the tool you would use and give a one-line reason: (a) whether a modal is open, (b) the product list returned by the API, (c) the logged-in user shared across the whole app.

SUBMIT YOUR QUICK-TEST ANSWERS INSIDE THE SAME WORD DOCUMENT

Place your quick-test answers in a clearly labelled “Quick Test” section after your Task 01–05 answers.

Submission Instructions

Prepare a single Microsoft Word document containing all of your work for this lab, and a ZIP of your project if your lecturer requests it.

Your Word document must include:

- Student name
- IT number
- Lab practical number (Lab 02)
- Screenshots of the completed React application (menu, search, dish detail, and the form with an error and a success state)
- Source-code snippets for the important components (DishCard, MenuList, useDebounce, useFetch, the form, and the routes)
- A brief explanation for each task (2–3 sentences)
- Your Quick Test answers (Q1–Q8)

Project ZIP (if required by the lecturer):

- Zip your project folder *without* the `node_modules` folder.

File naming and upload:

1. Rename the Word document using your IT number: **`ITXXXXXXXX_Lab02.docx`**
2. Example: **`IT22123456_Lab02.docx`**
3. If submitting the project, name the ZIP: **`ITXXXXXXXX_Lab02_Project.zip`**
4. Submit the Word document (and ZIP if required) to **CourseWeb** before the end of the 2-hour lab session.
5. **Late submissions will not be accepted** unless prior approval is given by the lecturer.

> **STUDENT INSTRUCTION**
>
> You must refer to the provided Lecture 02 material when completing the lab tasks. Your implementation and explanations should be based on the lecture content, class explanations, and the practical examples discussed during the lecture.

Evaluation Criteria (Marking Guide — 20 Marks)

For lecturer reference. Marks reward working code, correct React patterns, clean separation of concerns, and justified state choices.

Component	Marks	What earns full marks
Task 01 — Scaffold & structure	3	App runs via Vite; correct feature-based folders (grouped by domain, not file type).
Task 02 — Components, props, list	4	Reusable DishCard; MenuList maps with keys; props used correctly; conditional 'Sold out' and empty state.
Task 03 — State, hooks, events	4	Controlled search input; correct useDebounce hook (with cleanup); list filters on the debounced value.

Component	Marks	What earns full marks
Task 04 — Data fetching	4	useFetch returns {data, isLoading, error}; all three states rendered; data loads from /menu.json.
Task 05 — Routing & form	3	Working routes incl. /dish/:id (useParams) and 404; validated controlled form with per-field errors; LO4 table sensible.
Quick Test (Q1–Q8)	2	Q1–Q5 correct (b, c, b, c, b); Q6–Q8 show correct understanding.
Total	**20**	